

Train/Test Split

Group 5

2026-09-18

Table of contents

About this file	1
Loading the packages we need	2
File paths	2
1. Reading the analytical datasets	2
2. Target and class balance	3
3. Household-grouped train/test split	3
4. Checking the split for leakage	6
5. Saving the split datasets	7

About this file

This file covers the Milestone 2 rubric item “Modelling preparation & leakage control”: defining the modelling target, designing a train/test split and preventing leakage. The target, the predictors and the columns excluded for leakage were already decided in `Data Preprocessing.qmd` (sections 10.4–10.6) and documented in `Cleaned Datasets Data Dictionary.md` (sections 11–13), so all that is left to do here is split `model_data.csv` and `income_data.csv` into training and test sets.

`analytical_dataset.csv` is not split. It is the Power BI and reporting file, which the data dictionary describes as holding every working-age person for description rather than modelling.

Loading the packages we need

```
library(readr)
library(tidyverse)
```

File paths

```
analytical_directory <- "../Datasets/Analytical/"
```

1. Reading the analytical datasets

The split starts from the saved files rather than from objects left in memory by `Data Preprocessing.qmd`, so this file can be run on its own.

```
model_data <- read_csv(
  file.path(analytical_directory, "model_data.csv"),
  show_col_types = FALSE,
  col_types = cols(household_id = col_character())
)

income_data <- read_csv(
  file.path(analytical_directory, "income_data.csv"),
  show_col_types = FALSE,
  col_types = cols(household_id = col_character())
)
```

A CSV does not store factor levels, so the reference levels set in `Data Preprocessing.qmd` were lost when the files were saved. They are re-applied here, as described in `Cleaned Datasets Data Dictionary.md`, section 2.5. The categorical columns in `income_data` are converted back to factors.

```
model_data <- model_data |>
  mutate(
    employed = factor(employed, levels = c("Unemployed", "Employed")),
    education_band = fct_relevel(education_band, "Some secondary"),
    province = fct_relevel(province, "Gauteng"),
    population_group = fct_relevel(population_group, "Black African"),
```

```

    settlement_type = fct_relevel(settlement_type, "Metro urban"),
    edu_digital_group = fct_relevel(edu_digital_group, "Below matric, no internet"),
    smartphone_access = fct_relevel(smartphone_access, "None"),
    other_member_matric = fct_relevel(other_member_matric, "No")
  )

income_data <- income_data |>
  mutate(
    education_band = factor(education_band),
    population_group = factor(population_group),
    province = factor(province),
    settlement_type = factor(settlement_type)
  )

```

2. Target and class balance

Before splitting, the modelling target needs to be defined and checked for class imbalance. In `model_data.csv` the target is `employed` (Employed vs Unemployed). In `income_data.csv` the outcome is `log_salary`, which is continuous, so there is no class balance to check.

```
model_data |> count(employed) |> mutate(share = n / sum(n))
```

employed	n	share
Unemployed	10101	0.3514369
Employed	18641	0.6485631

There are 18641 employed people (64.9%) and 10101 unemployed people. This is a mild imbalance of roughly 2:1. It is not severe enough to need resampling such as SMOTE or down-sampling, and `glm()`, `rpart()` and `randomForest()` can all handle it directly. If a model struggles to pick up the smaller Unemployed class, Milestone 3 can still compare a class-weighted model with an unweighted one.

3. Household-grouped train/test split

Every row in both files is a person, and people in the same household share household-level predictors such as `any_home_internet`, `digital_access_score` and `has_fixed_internet` (section 10 of `Data Preprocessing.qmd`). A normal random split by row could put two

members of the same household on opposite sides, one in training and one in testing. The model would then learn that household's digital-access profile during training and be tested on a near-copy of it, which makes the test results look better than they really are.

To prevent this, the split is done by household instead of by person. 80% of the *distinct households* in each file are randomly chosen for training, and every person goes into the same set as the rest of their household.

```
set.seed(67) #Fixed seed so the split is reproducible

split_by_household <- function(data, train_share = 0.8) {
  households <- unique(data$household_id)
  train_households <- sample(households, size = round(train_share * length(households)))
  list(
    train = data |> filter(household_id %in% train_households),
    test = data |> filter(!household_id %in% train_households)
  )
}

model_split <- split_by_household(model_data)
model_train <- model_split$train
model_test <- model_split$test

income_split <- split_by_household(income_data)
income_train <- income_split$train
income_test <- income_split$test
```

The checks below confirm that every household is on only one side of the split, that the training and test sets do not share any households, and that the share of employed people stays close to the full dataset in both sets. Because the split samples households rather than people, the balance can shift slightly:

```
tibble(
  dataset = c("model_data", "income_data"),

  households_total = c(
    n_distinct(model_data$household_id),
    n_distinct(income_data$household_id)
  ),

  households_train = c(
    n_distinct(model_train$household_id),
    n_distinct(income_train$household_id)
  )
)
```

```

),

households_test = c(
  n_distinct(model_test$household_id),
  n_distinct(income_test$household_id)
),

household_overlap = c(
  length(intersect(model_train$household_id, model_test$household_id)),
  length(intersect(income_train$household_id, income_test$household_id))
),

rows_train = c(
  nrow(model_train),
  nrow(income_train)
),

rows_test = c(
  nrow(model_test),
  nrow(income_test)
),

rows_total_matches = c(
  nrow(model_train) + nrow(model_test) == nrow(model_data),
  nrow(income_train) + nrow(income_test) == nrow(income_data)
)
)

```

dataset	house- holds_to- tal	house- holds_train	house- holds_test	house- hold_over- lap	rows_train	rows_test	rows_to- testal_matches
model_data	16758	13406	3352	0	23041	5701	TRUE
in- come_data	13075	10460	2615	0	14371	3646	TRUE

```

model_train |>
  count(employed) |>
  mutate(share = round(n / sum(n), 3))

```

employed	n	share
Unemployed	8091	0.351
Employed	14950	0.649

```
model_test |>
  count(employed) |>
  mutate(share = round(n / sum(n), 3))
```

employed	n	share
Unemployed	2010	0.353
Employed	3691	0.647

The `model_data` training set has 13406 households (23041 people) and the test set has 3352 households (5701 people), with no households in common. The share of employed people stays close to the full dataset's 64.9% in both sets.

4. Checking the split for leakage

It is not enough to say that leakage was prevented; it needs to be shown. The columns that leak the outcome (salary, salary-based and reason-not-attending columns) were already removed in section 10.4 of `Data Preprocessing.qmd`, so this section checks that the split itself does not introduce any new leakage:

```
tibble(
  check = c(
    "No household appears in both model train and test",
    "No household appears in both income train and test",
    "model_data columns unchanged by the split",
    "income_data columns unchanged by the split"
  ),

  result = c(
    length(intersect(model_train$household_id, model_test$household_id)) == 0,
    length(intersect(income_train$household_id, income_test$household_id)) == 0,
    identical(names(model_train), names(model_data)) &&
    identical(names(model_test), names(model_data)),
    identical(names(income_train), names(income_data)) &&
    identical(names(income_test), names(income_data))
  )
)
```

```
)  
)
```

check	result
No household appears in both model train and test	TRUE
No household appears in both income train and test	TRUE
model_data columns unchanged by the split	TRUE
income_data columns unchanged by the split	TRUE

No household appears on both sides of either split, and the split neither adds nor removes any columns. `person_weight` and `household_id` stay in place as design columns rather than predictors (sections 11.1 and 12 of the data dictionary), so Milestone 3 can still apply the survey weights and identify each row's household without using either one as a predictor.

No further transformation is needed here. Encoding and the fixed centring of age were already done in `Data Preprocessing.qmd` before the split, and section 10.4 there explains why no scaling is needed. Those steps only use fixed rules or constants rather than values calculated from the data, so it makes no difference whether they happen before or after the split. Any future step that does calculate something from the data, such as filling in the 497 missing values in `matric_plus` or scaling a numeric predictor by its mean and standard deviation, belongs in Milestone 3. It should be fitted on `model_train` or `income_train` only, and then applied unchanged to the matching test set.

5. Saving the split datasets

The four split datasets are saved alongside the other analytical datasets in `Datasets/Analytical/`:

```
write_csv(  
  model_train,  
  file.path(analytical_directory, "model_train.csv"),  
  na = ""  
)  
write_csv(  
  model_test,  
  file.path(analytical_directory, "model_test.csv"),  
  na = ""  
)  
  
write_csv(  
  income_train,  
  file.path(analytical_directory, "income_train.csv"),  
  na = ""  
)  
write_csv(  
  income_test,  
  file.path(analytical_directory, "income_test.csv"),  
  na = ""  
)
```

```

income_train,
file.path(analytical_directory, "income_train.csv"),
na = ""
)

write_csv(
  income_test,
  file.path(analytical_directory, "income_test.csv"),
  na = ""
)

```

The files are then read back to confirm that each one has the same number of rows as the dataset in memory:

```

model_train_saved <- read_csv(
  file.path(analytical_directory, "model_train.csv"),
  show_col_types = FALSE,
  col_types = cols(household_id = col_character())
)

model_test_saved <- read_csv(
  file.path(analytical_directory, "model_test.csv"),
  show_col_types = FALSE,
  col_types = cols(household_id = col_character())
)

income_train_saved <- read_csv(
  file.path(analytical_directory, "income_train.csv"),
  show_col_types = FALSE,
  col_types = cols(household_id = col_character())
)

income_test_saved <- read_csv(
  file.path(analytical_directory, "income_test.csv"),
  show_col_types = FALSE,
  col_types = cols(household_id = col_character())
)

tibble(
  file = c(
    "model_train",
    "model_test",

```

```

    "income_train",
    "income_test"
  ),

  rows_in_memory = c(
    nrow(model_train),
    nrow(model_test),
    nrow(income_train),
    nrow(income_test)
  ),

  rows_in_saved_file = c(
    nrow(model_train_saved),
    nrow(model_test_saved),
    nrow(income_train_saved),
    nrow(income_test_saved)
  )
)

```

file	rows_in_memory	rows_in_saved_file
model_train	23041	23041
model_test	5701	5701
income_train	14371	14371
income_test	3646	3646

Four new files are saved in `Datasets/Analytical/`: `model_train.csv`, `model_test.csv`, `income_train.csv` and `income_test.csv`. `model_data.csv`, `income_data.csv` and `analytical_dataset.csv` stay unchanged. As with the other saved files, the reference levels from section 1 need to be re-applied after reading these files in Milestone 3.